RESEARCH NOTE

Protecting Structured Context

An Initial Before-and-After Study of JSON Integrity in Prompt Compression

Project: FocusedObjective / PromptCompression
Date: July 12, 2026
Evidence: 39 matched FocusFit prompt pairs

Abstract

Token-level prompt compression can reduce cost and context size, but applying a learned compressor directly to JSON creates a disproportionate integrity risk: removing one field name, identifier, sign, value, or structural token can change the meaning of an otherwise readable prompt. PromptCompression recently changed its pipeline so valid JSON is handled deterministically—converted to TOON when safe and beneficial, otherwise protected from LLMLingua-2—and only explicitly allowlisted long string values may be compressed under a tenant policy.

The improvement was developed through a five-step empirical loop documented first in the *Information-Loss Evaluation Report: Original vs. Compressed Prompts — First Five Benchmark Records*. That report created five task-specific evaluation questions for each of five prompt pairs and inspected whether the compressed context retained the facts required to answer them. Two of five pairs failed the proposed fidelity gate at high severity, two passed, and one was conditional. The most important failure removed JSON field names while retaining their values, destroying the numeric-label relationship required by the task.

The failure was traced to small JSON blocks that were not eligible for deterministic TOON conversion and could therefore reach learned compression. The pipeline was changed to protect valid JSON regardless of size, and the prompts and integrity checks were rerun. This targeted questioning exposed a local key–value association failure that aggregate whole-prompt metrics did not reveal.

This research note also compares 39 byte-identical production-like FocusFit prompts processed before and after JSON protection. The protected version retained all 4,103 row-level distinct numeric literals and all 621 UUIDs observed in the original prompts. The earlier version retained 4,094 numeric literals (99.78%) and 616 UUIDs (99.19%). All 1,878 row-level distinct JSON field-name labels were discoverable somewhere in both versions. That apparently reassuring aggregate result demonstrates the limitation of global token-presence metrics: a field name can occur elsewhere in a long prompt while being absent from the specific JSON object where its relationship to a value matters.

The integrity improvement cost 291 additional compressed tokens across 318,770 original tokens. Weighted reduction moved from 5.369% to 5.278%, a decrease of 0.091 percentage points. These results support JSON protection as a favorable integrity–compression trade. They remain an initial

result: the CSVs support literal coverage checks but not a complete typed round-trip proof of every JSON object.

1. Why JSON needs a different compression policy

Natural-language prose often contains removable redundancy. JSON is different. Its field names, scalar types, array membership, ordering, and punctuation encode relationships that downstream tools or models may interpret precisely. A compressed sentence can remain understandable after dropping a modifier; a JSON record can silently change meaning after dropping a key, a negative sign, one repeated array item, or a Boolean value.

Raw LLMLingua-2 treats prompt compression as token classification. That is useful for eligible prose, but it does not inherently guarantee preservation of every structured field required by a particular application. PromptCompression therefore treats learned compression as one stage inside a larger deterministic pipeline rather than applying it indiscriminately to the entire prompt.

2. The protection change

The current pipeline applies the following policy to structured JSON:

- Detect valid JSON objects and arrays before learned compression.
- Protect small JSON and exactness-sensitive JSON verbatim.
- Protect JSON Schema examples, exact fixtures, duplicate-key JSON, and LLM tool exchanges.
- Convert eligible medium or large JSON to TOON only when the conversion passes safety and savings gates.
- Replace the protected structured segment with a placeholder before any outer LLMLingua-2 call, then restore it afterward.
- Allow model compression inside JSON only for tenant-authorized string paths, with limits on value size, count, and maximum reduction.
- Reject and restore any selectively compressed string that loses protected URLs, emails, numbers, money values, identifiers, code, or supported constraints.

The key architectural improvement is not simply “keep more tokens.” It separates structured representation from probabilistic prose compression. Crucially, JSON size now determines whether TOON conversion is worth attempting—not whether the JSON receives protection. Small valid JSON that is not TOON-compressed is still protected from the learned compression stage. Field names, types, arrays, and unapproved values remain under deterministic control.

3. Study design

3.1 Initial comparison: the first-five information-loss report

The initial comparison examined the first five benchmark prompt pairs, with observed token reductions ranging from 13.2% to 21.0%. It defined five evidence-based questions per prompt—25 probes in total—and listed the facts each response would need to include.

The report's pair-level findings were:

- **Prompt 1 — FAIL, high severity.** The compressed prompt retained values but removed field names including `verificationFrictionScore`, `changeSprawlScore`, `reviewContentionScore`, `mergeFrictionRisk`, `pendingCount`, and `rerunCount`. Without those labels, the values could not be mapped reliably to their meanings.
- **Prompt 2 — PASS, low severity.** Decision-relevant metadata and explicit evidence gaps remained available, although identifier formatting degraded.
- **Prompt 3 — FAIL, high severity.** The decision qualifier “primarily” and the central evidence “very quickly with few commits” were weakened or removed.
- **Prompt 4 — PASS, low severity.** Scope, priorities, schedule, threshold, channel decision, and open questions remained available.
- **Prompt 5 — CONDITIONAL, medium severity.** Core summary facts survived, but model/issue provenance and the `Composite` score label were corrupted.

The cross-prompt conclusion identified label deletion as the most serious failure mode: preserving a number without its field name can be worse than deleting the number because it invites confident misattribution. The report recommended 100% accuracy for critical probes and numeric-label pairing, alongside task-aware measures for qualifiers, negation, overrides, and exact identifiers.

3.2 Failure discovery and remediation process

The JSON-protection change followed a reproducible five-step process:

1. **Capture before-and-after samples.** Original prompts and their compressed outputs were exported so information removal could be inspected directly.
2. **Ask questions that depend on prompt evidence.** The first-five report created five questions per prompt, including questions whose answers required particular fields, values, qualifiers, and relationships. The original and compressed contexts were checked for whether each required answer remained recoverable.
3. **Diagnose the failure.** The report found that field names inside small JSON blocks could be removed by learned compression. In Prompt 1, multiple friction, risk, CI, and rerun values survived without their labels. When a field name disappeared, the associated value lost its meaning or became unavailable to the downstream task, even if much of the surrounding prompt remained readable.
4. **Change the protection boundary.** The pipeline was updated so all detected valid JSON is protected from the outer LLMLingua-2 stage, including JSON too small to qualify for

deterministic TOON conversion. Eligible larger JSON can still be converted to TOON; small or otherwise ineligible JSON remains protected in JSON form.

5. **Rerun and retest.** The same prompts were compressed again with protection enabled and the question-based integrity checks were repeated to verify that the previously observed loss no longer occurred.

This sequence is important because it links an observed downstream information failure to a specific compression behavior, implements a bounded fix, and checks the fix on matched inputs. It is stronger than tuning against token savings alone.

3.3 Paired rerun dataset

The comparison uses the FocusFit benchmark exports immediately before and after the JSON-protection release. Both files contain 39 rows. Pairing on `callId` and `messageIndex` produced 39 matched prompts, and the `originalText` value was byte-identical for every pair.

The paired originals contained 318,770 estimated tokens. Fourteen compressed outputs changed after protection; 25 were identical. Every row reported `model_force`, although the learned model ran on only 15 of the 39 rows because the remaining prompts followed an unchanged path.

3.4 Integrity diagnostics

For every prompt pair, the analysis extracted:

- quoted JSON-style field-name labels from the original prompt;
- distinct numeric literals, including explicit signs and percentages;
- UUIDs; and
- original and compressed token counts.

A label or literal was counted as retained when the exact value remained discoverable in the compressed prompt. Field-name matching allowed the label to appear in JSON or a transformed structured representation such as TOON.

This is a conservative diagnostic, not a complete semantic or structural proof. Exact matching can overstate loss when formatting changes, and whole-prompt label presence cannot prove that a field remains associated with the correct value or appears with the correct multiplicity. The targeted test questions were therefore the decisive diagnostic for the small-JSON field-name failure.

4. Results

Metric	Before JSON protection	After JSON protection	Difference
Matched prompts	39	39	—
Original tokens	318,770	318,770	0
Compressed tokens	301,654	301,945	+291

Metric	Before JSON protection	After JSON protection	Difference
Tokens saved	17,116	16,825	−291
Weighted token reduction	5.369%	5.278%	−0.091 pp
Distinct numeric literals retained	4,094 / 4,103 (99.78%)	4,103 / 4,103 (100.00%)	+9 instances
UUIDs retained	616 / 621 (99.19%)	621 / 621 (100.00%)	+5 instances
Distinct field-name labels discoverable	1,878 / 1,878 (100.00%)	1,878 / 1,878 (100.00%)	No observed unique-label loss in either run

4.1 Integrity improved at negligible compression cost

The protected pipeline recovered every distinct numeric literal and UUID that the strict matcher found missing in the earlier compressed outputs. Recovered numeric examples included explicitly negative values, a category where losing the sign can reverse meaning.

The cost was small: 291 tokens across a 318,770-token input corpus, or fewer than one additional retained token per thousand original tokens. The relative token savings decreased by approximately 1.7%, but the absolute reduction changed by only 0.091 percentage points.

4.2 What the field-name result does—and does not—show

The automated benchmark found 100% row-level distinct field-name label coverage both before and after protection. Taken alone, that result would suggest there was no field-name problem. The question-based review showed why that conclusion would be wrong: a field label could remain somewhere else in a long tool history while being removed from the small JSON object whose local key–value association was required to answer the test question.

The improvement is therefore both behavioral and architectural. The targeted questions exposed a loss of relevant prompt data; after the protection boundary was expanded and the prompts were rerun, the same test process no longer observed that failure. Architecturally, detected JSON structure is now excluded from the outer probabilistic model call whether or not the block is large enough for TOON. Field names, types, arrays, and unapproved values are controlled by deterministic parsing and reconstruction.

The whole-prompt CSV cannot prove that every repeated field occurrence, type, array position, or key–value association survived. JSON may also be safely represented as TOON, making byte-for-byte JSON comparison inappropriate. A stronger test should decode every protected block and compare its typed data model with the original.

5. Recommended confirmatory benchmark

The larger production run should retain the current CSV fields and add segment-level integrity evidence:

- a stable `promptPairId` and compressor commit/version;

- the detected structured-segment type and applicable policy;
- a canonical hash of the parsed original JSON data model;
- a canonical hash after decoding the final JSON or TOON representation;
- field count, array count, scalar count, and maximum nesting depth before and after;
- exact key-set, type, value, array-order, and occurrence-count checks;
- a record of whether each segment was protected, converted to TOON, minified, selectively compressed, or restored by a safety fallback; and
- downstream question-answer tests for values, relationships, constraints, and negative controls.

The primary structural release gate should be zero typed round-trip failures. Compression rate should remain a secondary optimization subject to that gate.

6. Limitations

- The study contains 39 prompts from one workload.
- The field-name metric measures discoverability, not full key–value association or multiplicity.
- The initial question set and manual pass/fail observations were not exported as structured benchmark rows, so this note documents the process and qualitative result without claiming a quantitative answer-accuracy improvement.
- Exact literal matching can classify harmless formatting changes as losses.
- The exports do not contain decoded JSON/TOON trees or canonical structural hashes.
- The study does not include downstream model answers or human semantic judgments.
- Twenty-five of the 39 outputs were unchanged, limiting the number of prompts on which the protection change could have an observable effect.

7. Conclusion

The initial before-and-after work supports JSON protection as a high-value safety improvement. The first-five information-loss report established the baseline: two of five prompts had high-severity losses, and Prompt 1 lost multiple decision-critical field names while retaining their values. Those 25 task-specific probes turned a readable-looking compressed prompt into a concrete failure diagnosis. The pipeline was changed so all valid JSON receives protection even when it is too small for TOON, and the prompts and tests were rerun. In the subsequent 39 paired export rows, observed distinct numeric-literal and UUID retention reached 100% while weighted token savings decreased by only 0.091 percentage points.

The result should be preserved as an engineering milestone, not presented as a complete proof of lossless structured compression. The next larger benchmark should add typed round-trip hashes and downstream question answering. If those tests remain clean at scale, the project will be able to make a substantially stronger claim: deterministic JSON handling improves prompt integrity without materially compromising the savings delivered by the broader compression pipeline.

References

1. Pan, Z., et al. “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression.” Findings of ACL, 2024. https://aclanthology.org/2024.findings-acl.57/

2. FocusedObjective. “PromptCompression.” Source repository. https://github.com/FocusedObjective/PromptCompression

3. FocusedObjective. “Tagged JSON Compression.” https://github.com/FocusedObjective/PromptCompression/blob/main/docs/tagged-json-compression.md

4. FocusedObjective. *Information-Loss Evaluation Report: Original vs. Compressed Prompts — First Five Benchmark Records.* Internal benchmark report, July 2026.